Rapport de stage

Refonte des outils d'annotation et amélioration de la chaîne d'intégration continue chez Viziosense

Evgenii Kniazev

14 novembre 2025

Ce rapport présente mon stage au sein de **Viziosense**, une entreprise de vision par ordinateur embarquée (edge) spécialisée dans la mesure de flux, la gestion de stationnement et la densité de foule, dans le respect de la vie privée. Ma mission principale a consisté à **refondre les outils d'annotation** utilisés par les intégrateurs afin d'améliorer l'UX, la robustesse et la productivité lors des déploiements. J'ai également **optimisé la chaîne CI/CD** des projets périphériques (interface locale appareil) et livré plusieurs **améliorations de sécurité et d'expérience utilisateur**.

Table des matières

1 L'entreprise **2**
1.1 Mission et proposition de valeur . . . 2
1.2 Modèle de déploiement et écosystème . . . 2
1.3 Organisation . . . 2
1.4 Panorama des produits . . . 3

2 Ma mission principale : refonte des outils d'annotation **3**
2.1 Contexte et état initial . . . 3
2.2 Analyse d'usage a posteriori . . . 4
2.3 Approche et architecture . . . 6
2.4 Approche produit et démarche centrée utilisateur . . . 7
2.5 Ce qui se passe à l'écran . . . 8
2.6 Comment on travaillait au quotidien . . . 9
2.7 Ce que ça a changé . . . 9

3 Autres tâches accomplies **10**
3.1 Interface locale appareil (Go + React) : sécurité et UX . . . 10
3.2 CI/CD et performance de build (GitLab CI, Docker) . . . 10
3.3 Écosystème intégrateur . . . 11

4 Compétences développées **11**
4.1 Techniques . . . 11
4.2 Méthodologiques et humaines . . . 11

5 Conclusion **11**

1 L'entreprise

1.1 Mission et proposition de valeur

Fondée en 2022, **Viziosense** conçoit des *capteurs vision* (caméras avec processeur et NPU) capables d'analyser les images directement en bordure de réseau (*on-edge*). L'objectif est de fournir, en temps réel et **sans recourir à de la vidéosurveillance intrusive**, des *données opérationnelles* — par exemple :

- **Stationnement** : quelles places sont libres/occupées sur un parking (*VizioPark*) ;
- **Comptage par lignes/zones** : combien de personnes, vélos, voitures traversent des barrières virtuelles ou se trouvent dans des zones (*VizioCount*) ;
- **Densité de foule** : estimation du nombre de personnes dans des aires données via un modèle « têtes » avec **bornes min/max** (*VizioCrowd*) ;
- **Logistique de yard** : fluidifier l'accueil et l'orientation des camions, réduire les temps d'attente et les frictions (*VizioYard*).

Les traitements s'effectuent sur site, seuls des *événements* et *agrégats* anonymes sont émis (par exemple, état d'une place, passage sur une ligne, densité estimée). Cette approche concilie **utilité opérationnelle** et **respect de la vie privée**.

FIGURE 1 – Capteur à deux cameras.

1.2 Modèle de déploiement et écosystème

Viziosense opère principalement en **B2B2B** : l'entreprise équipe et accompagne des **intégrateurs** qui déploient ensuite les solutions auprès de leurs clients finaux (collectivités, industriels, gestionnaires de sites). Les intégrateurs sont les interlocuteurs directs de Viziosense : ils installent les capteurs, paramètrent les zones d'analyse et relient les données aux systèmes locaux. Les utilisateurs finaux, eux, bénéficient des données issues des capteurs pour améliorer la circulation, optimiser le stationnement ou fluidifier les opérations sur site.

Parmi les déploiements récents : réseaux de transport (SNCF, CFL, Engie), aéroports et ports, villes et collectivités (MEL, Reims, Beauvais, Ettelbruck, Differdange), ainsi que plusieurs sites industriels (FM Logistic, Coca-Cola, Renault).

1.3 Organisation

L'équipe compte **environ 15 personnes**, majoritairement côté technique (edge, back, front, ML, embed). La direction pilote la feuille de route produit, les ventes structurent les

offres avec les intégrateurs, et l'équipe d'intégration garantit la qualité des déploiements (positionnement capteurs, paramétrage, validation).

J'ai été encadré par **Alessandro Osias**, ingénieur en vision par ordinateur et développeur full-stack, avec qui j'ai échangé régulièrement sur les aspects liés à l'**annotation** — logique Canvas, ergonomie de l'outil, interactions avec le back-end. En parallèle, j'ai bénéficié du mentorat de **Jimmy Lucidarme**, notre tech lead, plutôt orienté **back-end et IoT**. Il m'a accompagné sur des sujets plus techniques : corrections de fonctions **Go** et **AWS Lambda**, revue de code approfondie et bonnes pratiques de structuration serveur.
Cette double supervision a été particulièrement formatrice : Alessandro apportait la vision produit et usage terrain, tandis que Jimmy garantissait la robustesse technique et la cohérence du code à l'échelle du projet.

1.4 Panorama des produits

Produit	**Finalité**	**Entrées**	**Sorties (événements)**
VizioPark	Gestion de places (libre/occupée), guidage	Flux caméra + annotation *spots*	État spot, changement d'état
VizioYard	Orchestration camions & quais	Vidéo + intégrations SI	État spot, changement d'état
VizioCount	Comptage par *lignes* (barrières) et *zones*	Flux caméra + lignes/polygones	Passages par direction, présence par zone
VizioCrowd	Estimation de densité de foule	Flux caméra + zones	Compte estimé (min/avg/max)

2 Ma mission principale : refonte des outils d'annotation

2.1 Contexte et état initial

Au départ, l'annotation servait de mode d'emploi visuel pour chaque caméra : on indiquait où se trouvaient les **places** de parking, où tracer les **lignes/barrières** pour le comptage, et quelles **zones/polygones** représentaient la densité. La première version de l'outil, construite sur **Canvas + FabricJS**, remplissait cette fonction de base, mais elle montrait rapidement ses limites. Impossible d'éditer correctement un tracé, aucun aperçu avant validation, et un fort mélange entre logique et affichage. Résultat : beaucoup d'allers-retours, des erreurs fréquentes et une perte de temps dès qu'une caméra était déplacée ou qu'il fallait réaligner les zones.

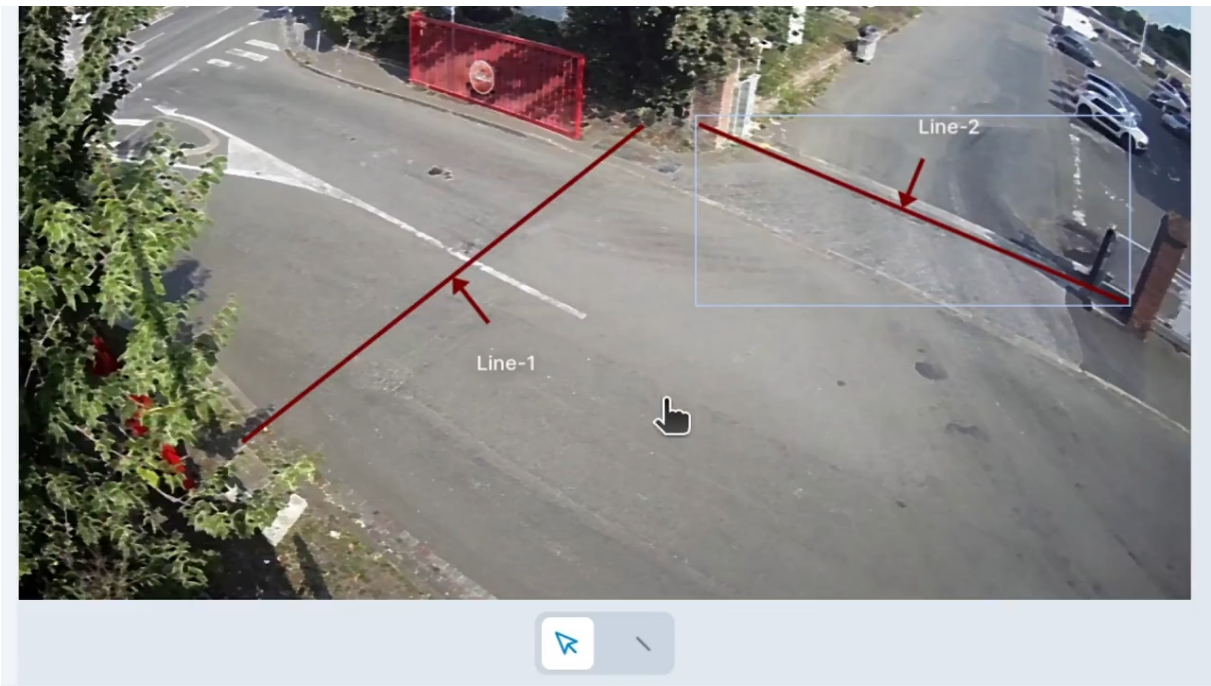

FIGURE 2 – Exemple d'**annotation de barrières et zones** dans la version finale de l'outil.

Ce que cela impliquait pour les intégrateurs

- **Pas d'édition** — toute modification nécessitait de supprimer et recréer un élément complet.
- **Caméra déplacée = tout refaire** — l'overlay devenait faux, obligeant à repartir de zéro.
- **Couplage fort entre UI et logique** — en pratique, cela signifiait aussi l'absence d'outils de précision : **pas de zoom local**, ni de **repérage magnétique (snap)** sur les arêtes ou points existants. Les intégrateurs devaient ajuster les tracés « à la main », parfois au pixel près, ce qui augmentait la charge mentale et le risque d'erreurs sur des scènes complexes.

2.2 Analyse d'usage a posteriori

En milieu de stage, alors que la refonte technique était déjà bien engagée, j'ai voulu **comprendre le parcours réel des utilisateurs** avec l'ancienne version. Nous avons donc exploité les **journaux applicatifs anonymisés**, retraçant toutes les sessions de l'été : chaque visite était enregistrée avec son `deviceId`, le type de page (annotation, configuration, logs...), la durée et un horodatage.
Les données ont été nettoyées et visualisées dans des notebooks Python (`pandas + Plotly`), afin de reconstituer le comportement des intégrateurs au fil de leurs visites.

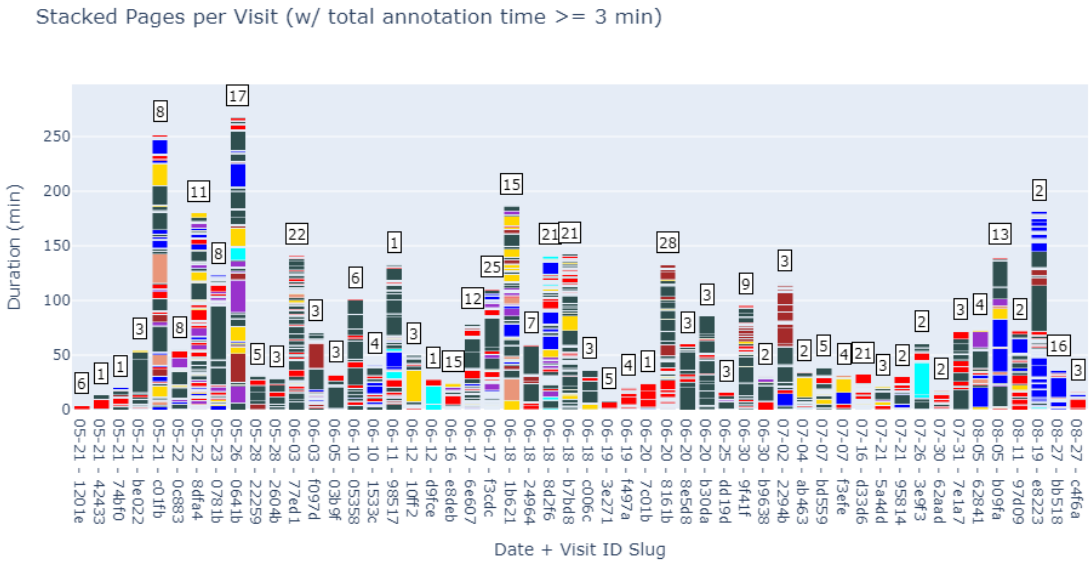

FIGURE 3 – Empilement des pages par visite (sessions d'annotation d'une durée supérieure ou égale à 3 minutes). Chaque couleur correspond à un type de page (annotation, journaux, configuration, etc.). Les chiffres indiquent le nombre d'appareils annotés pendant la session.

La première visualisation, en barres empilées, illustre la **navigation séquentielle** des intégrateurs. On y voit des allers-retours constants entre les pages d'annotation et celles de configuration — voire des passages fréquents par les journaux de logs. Ce schéma traduisait une fragmentation du travail : chaque opération nécessitait de changer de contexte, ce qui rallongeait le temps global d'annotation.

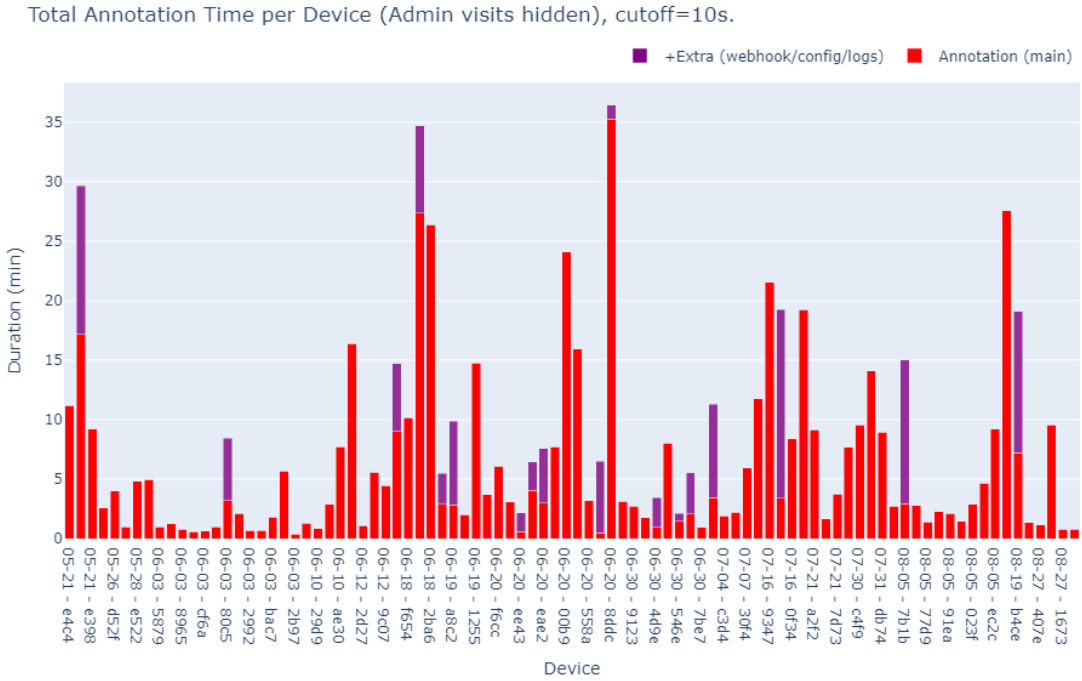

FIGURE 4 – Durée totale d'annotation par appareil (*admin visits hidden, cutoff = 10s*). En rouge : temps passé sur les pages d'annotation principales ; en violet : activités annexes (webhook, configuration, logs).

En agrégeant les durées par appareil, on a observé des écarts considérables. Certaines sessions dépassaient 25 à 30 minutes, souvent à cause d'allers-retours répétés entre annotation et configuration. Les pics traduisaient clairement les zones de friction que la nouvelle version devait adresser.

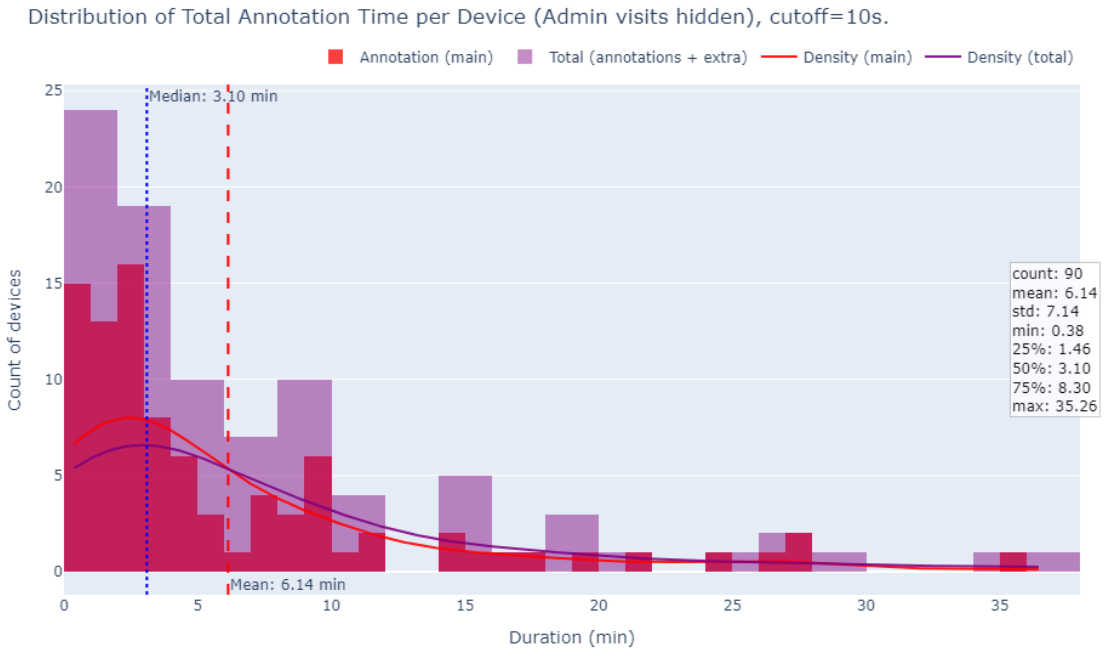

FIGURE 5 – Distribution du temps total d'annotation par appareil (*admin visits hidden, cutoff = 10s*). En rouge : temps passé sur les pages d'annotation principales ; en violet : total incluant les activités annexes (webhook, configuration, logs). Les lignes pointillées indiquent la moyenne et la médiane.

Cette visualisation met en évidence la **répartition des durées d'annotation** sur l'ensemble du parc de caméras. La majorité des appareils se situent sous la barre des **3 à 5 minutes**, mais la moyenne s'étire autour de 6 minutes, avec quelques valeurs extrêmes dépassant 30 minutes. La courbe violette (densité totale) montre une longue traîne — typique d'un flux où quelques sessions concentrent la plupart des frictions.
En pratique, cela veut dire que pour la majorité des intégrateurs, l'outil fonctionnait « assez bien », mais dès que la scène devenait complexe (zones nombreuses, caméra légèrement décalée, reconfigurations multiples), le temps explosait. C'est précisément ce type de dispersion que la refonte visait à réduire : rendre le processus plus homogène, limiter les interruptions et éviter les reprises complètes d'annotation après un simple ajustement.

2.3 Approche et architecture

Nous avons suivi une organisation « *MVC* pragmatique ». Le **modèle**, côté serveur, reste la *source de vérité* : chaque place/ligne/zone a un **UUID** unique, une **géométrie normalisée** en $[0, 1]$ (indépendante de la résolution), un *label* et une *revision*. La **vue**, c'est le Canvas/FabricJS : un calque interactif où l'on dessine, sélectionne et manipule. Le **contrôleur** (services Angular) fait la liaison : il charge, valide, calcule ce qui a changé et synchronise avec le serveur.

Pourquoi ça compte, concrètement

— Si l'image change de taille ou si la caméra bouge un peu, **on réaligne** sans tout casser.
— Les **UUID ne changent pas** : l'historique et les tableaux de bord restent cohérents.
— La **logique métier** n'est plus noyée dans l'affichage : on peut ajouter des fonctions sans tout repriser.

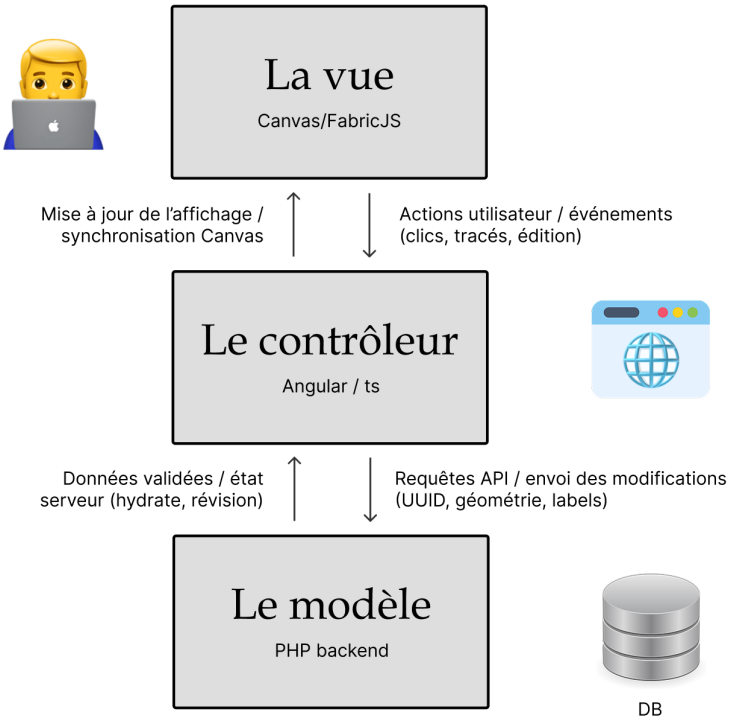

FIGURE 6 – Schéma MVC pragmatique (modèle / vue / contrôleur).

2.4 Approche produit et démarche centrée utilisateur

Avant de plonger dans le code, j'ai pris un peu de recul pour réfléchir à **ce que les utilisateurs essaient réellement d'accomplir**. Dans la logique du *Jobs To Be Done* (JTBD), il ne s'agissait pas seulement de « tracer une ligne » ou « placer une zone », mais plutôt de **mettre en place un dispositif de comptage fiable le plus vite possible**, sans craindre de devoir tout recommencer. Ce changement de perspective m'a aidé à aborder la refonte non pas comme une suite de fonctionnalités à ajouter, mais comme une réponse à un besoin concret de fluidité et de confiance dans l'outil.

Phase exploratoire

— J'ai d'abord esquissé plusieurs idées sur papier — comment rendre le traçage plus lisible, la correction plus directe, et l'expérience plus fluide.
— Ensuite, j'ai échangé avec l'un de nos **développeurs seniors**, également utilisateur régulier de l'outil, pour observer sa manière de s'en servir et comprendre ses irritants réels.
— À partir de ce retour, j'ai conçu un **prototype Figma** intégrant plusieurs hypothèses d'amélioration : aperçu dynamique, duplication rapide, et numérotation automatique des labels.

Validation interne et test UX

— Nous avons mené un premier test en interne : le prototype a mis en lumière plusieurs points faibles (notamment des zones confuses à l'écran) qui ont été laissés de côté pour ne pas alourdir l'interface. L'accueil global a néanmoins confirmé que la nouvelle logique ferait **gagner beaucoup de temps** aux intégrateurs.
— Après la phase de développement et la mise à jour sur l'environnement de **staging**, nous avons organisé un **test utilisateur externe** avec l'un de nos clients. Le format suivait la méthode du *think-aloud* : l'utilisateur devait accomplir une tâche réelle en verbalisant ce qu'il voyait, pensait et envisageait de faire.

Le test a montré que les changements étaient perçus comme positifs et cohérents — certains éléments avaient enfin « du sens » à l'usage. D'autres points ont toutefois révélé un besoin de **guidage visuel supplémentaire** (par exemple, des labels expliquant les nouvelles actions possibles). Ces retours ont été intégrés dans la version finale avant déploiement. En résumé, cette approche a permis de combiner la **rigueur technique** du projet avec une **démarche produit centrée sur les usages réels**.

2.5 Ce qui se passe à l'écran

À l'usage, l'outil devient plus direct et plus précis. On **voit** le segment avant de cliquer, on **déplace** un point au pixel près grâce à une **loupe** $\times 3$, on **inverse** le sens d'une ligne, on **duplique** quand la scène se répète. Pour les parkings, la saisie **suggère la suite logique des labels** (`P0001`, `P0002`, ...) comme dans un tableur. Tout cela réduit la fatigue cognitive et les petites erreurs qui coûtent du temps plus tard.

Fonctions utiles ajoutées

— **Prévisualisation en temps réel** du tracé (curseur et segments fantômes).
— **Édition complète** des objets (déplacer, redimensionner, inverser, dupliquer).
— **Saisie assistée des labels** (auto-incrément cohérent).
— **Loupe de précision** pour les points délicats (toits, angles, bordures).
— **Validations géométriques** en direct (pas de polygones auto-croisés, lignes non dégénérées).

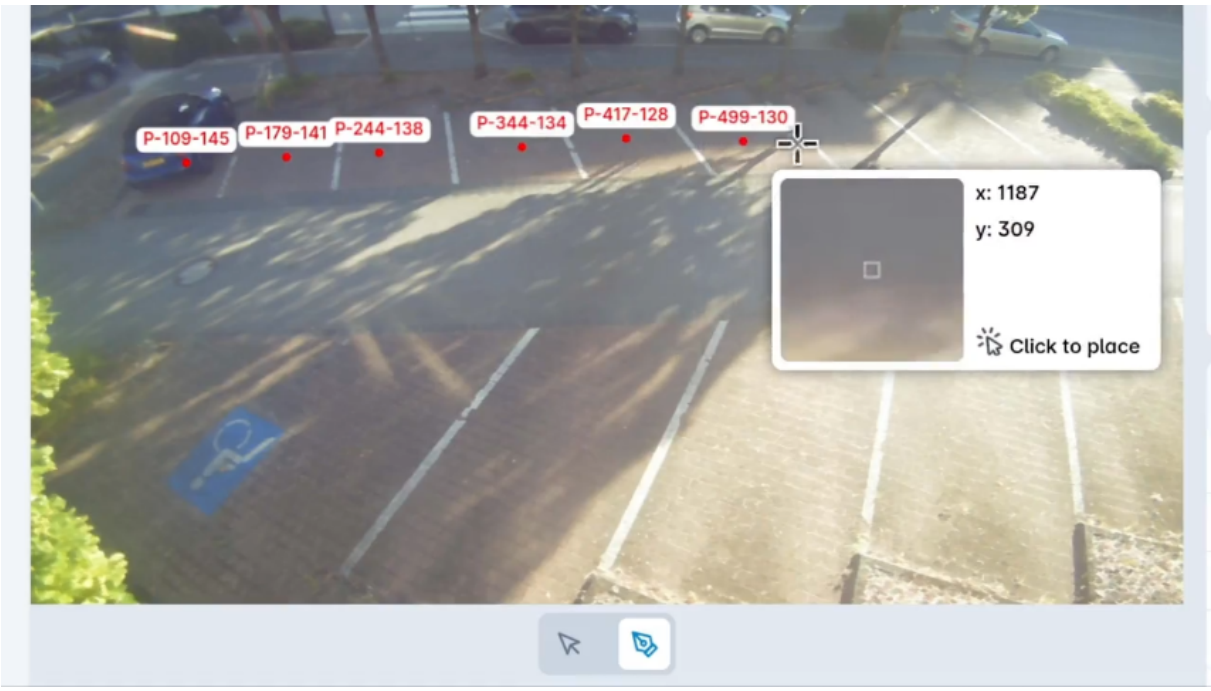

Figure 7 – Prévisualisation et loupe de précision lors du tracé.

2.6 Comment on travaillait au quotidien

Nous avons suivi une discipline simple, qui marche bien en équipe. Pour chaque fonctionnalité, je créais une **branche** dédiée depuis `dev` (par ex. `VSDEV-123-descr`). J'avançais par **petits commits**, et je **rebasais** régulièrement pour rester à jour avec `dev`. Ensuite, j'ouvrais une **Merge Request** : description claire, captures si besoin, et **revue de code** par un pair. Une fois validée, on fusionnait dans `dev` et on **déployait sur `staging`** pour tester en conditions proches de la production.

Pourquoi cette rigueur

- **Sécurité** : les relectures attrapent les erreurs tôt.
- **Prévisibilité** : `staging` évite les surprises en prod.
- **Traçabilité** : chaque changement est relié à un ticket (`VSDEV-123`), donc explicable.

2.7 Ce que ça a changé

Au final, on **annote plus vite** et **mieux**. On **voit** ce qu'on fait, on **corrige** sans tout refaire.

Pour objectiver ces gains, nous avons suivi les **données d'usage réelles** via notre instance Matomo. La capture ci-dessous (début octobre) illustre l'activité sur les outils d'annotation : les événements `click`, `toggle`, `create`, `delete`, et `suggestion_accepted` y figurent désormais de manière distincte. Cela montre que les intégrateurs utilisent effectivement les nouvelles fonctions — en particulier la **saisie assistée** (suggestions automatiques de labels) et les **raccourcis clavier**, jusque-là absents.

Event Categories

EVENT CATEGORY	EVENT ACTION	VISITS	EVENTS	EVENT VALUE	AVERAGE VALUE	MINIMUM EVENT VALUE	MAXIMU
PaginateDashboard	entry	305	946	-	-	-	
Image	Request	256	1,300	1,300	1	1	
AnnotationTools	delete	55	470	-	-	-	
Device	Soft-reboot	36	68	68	1	1	
AnnotationTools	click	34	113	-	-	-	
AnnotationTools	toggle	31	678	656	0.97	-	
AnnotationTools	start	27	50	120	2.4	1	
AnnotationTools	create	22	480	-	-	-	
AnnotationTools	hotkey_used	9	27	-	-	-	
AnnotationTools	suggestion_accepted	7	129	373	2.89	1	
AnnotationTools	cancel	4	7	-	-	-	
Fleets	toggle	4	4	4	1	1	
AnnotationTools	flipped	2	6	-	-	-	
Device	reboot	1	1	1	1	1	
Totals	-	**793**	**4,279**	**2,522**	**1.13**	-	

FIGURE 8 – Extrait du tableau de bord Matomo (6 octobre). Les événements liés aux outils d'annotation (*AnnotationTools*) traduisent l'usage concret des nouvelles fonctionnalités : création, suppression, bascule, acceptation de suggestions, etc.

En somme, la refonte n'a pas simplement accéléré l'annotation : elle a rendu le processus **plus lisible**, **moins fragile**, et surtout **plus mesurable**.

3 Autres tâches accomplies

3.1 Interface locale appareil (Go + React) : sécurité et UX

- **Changement de mot de passe** par l'utilisateur lors de l'initialisation (remplacement d'un mot de passe par défaut commun, amélioration sécurité).
- Ajustements UX et messages d'erreur pour un **onboarding** plus fluide (réseau, LTE/4G, diagnostics).

3.2 CI/CD et performance de build (GitLab CI, Docker)

Dans un projet combinant un **frontend React (TS)** et un **backend Go**, les déploiements fréquents deviennent vite un point sensible. Chaque changement, même minime, déclenchait une reconstruction complète de l'image Docker, un enchaînement long et coûteux (environ 40 minutes pour un cycle complet). L'objectif a donc été de **rendre les builds plus rapides et plus prévisibles**, sans compromettre la reproductibilité.

1. Conteneurisation et principe de base
Chaque composant du projet tourne dans un **conteneur Docker** — une sorte de boîte hermétique qui embarque tout ce qu'il faut pour exécuter l'application, de la même manière sur n'importe quelle machine. Le backend Go, le frontend compilé et les outils de configuration sont chacun construits dans une image distincte, puis orchestrés par GitLab CI pour être testés, assemblés et déployés.

2. Optimisation des builds multi-étapes
Nous avons mis en place des **Dockerfiles multi-étapes** séparant la phase de compilation (*builder*) et l'image d'exécution (*runtime*) :

- Pour le backend Go : téléchargement des dépendances avec `go mod download` en couche indépendante, compilation avec options d'allègement, puis copie du binaire final dans une image *alpine* minimale.
- Pour le frontend React : installation, compilation en artefacts statiques, puis transfert vers l'image d'exécution.

Ces séparations permettent de **réutiliser les couches de build** dès lors que les dépendances n'ont pas changé. Cependant, un point bloquant persistait : le **backend Go servait directement les fichiers compilés du frontend**. Or, chaque recompilation du frontend modifiait les métadonnées du dossier (*hash*), ce qui invalidait le cache Docker, même si le code Go restait identique.

3. Réorganisation pour maximiser le cache
Pour résoudre ce problème, nous avons dissocié le moment où le dossier `/dist` (frontend compilé) est copié dans l'image backend :

- Le backend est désormais construit et mis en cache avant la copie du `dist`.
- La phase de copie du frontend est déplacée à la fin du Dockerfile et ne provoque plus la reconstruction complète du backend.

Cette simple réorganisation a permis de **préserver le cache des couches lourdes** — en particulier la compilation Go — et de ne reconstruire que ce qui change réellement.

4. Résultat et impact concret
Grâce à cette optimisation, le **temps moyen de pipeline a été divisé par deux**, passant d'environ 40 minutes à 20 minutes dans les cas courants. Les itérations de test se sont donc accélérées de manière significative. Concrètement, cela change tout pour un développeur : si l'on expérimente par exemple une nouvelle gestion GPIO sur un appareil, on peut désormais tester **deux fois plus de versions par heure**. L'équipe gagne ainsi

en réactivité et peut valider des hypothèses techniques sans attendre un cycle complet de build à chaque essai.

5. Industrialisation et cohérence
Enfin, nous avons introduit un **Makefile** unifié pour standardiser les cibles locales et CI :
- `make build` → construit l'image locale avec les bons arguments de cache ;
- `make deploy` → déclenche la publication vers le registre GitLab.

Cette harmonisation a simplifié la collaboration entre développeurs et stabilisé la chaîne CI/CD. Les builds sont plus **rapides**, **déterministes** et **moins sujets aux erreurs**, ce qui permet de concentrer l'effort sur la valeur ajoutée — le code, pas les pipelines.

3.3 Écosystème intégrateur

- Contributions à l'**onglet Marqueurs** côté intégrateur (meilleure visibilité des lignes/zones/spots à l'échelle flotte).
- Rédaction/édition de **documentation** (procédures d'annotation, bonnes pratiques de tracé et de vérification).

4 Compétences développées

4.1 Techniques

- **Front** : Angular (plateforme intégrateur), React (service local), TypeScript, FabricJS, Canvas HTML5, gestion d'état et de modèles.
- **Back & tooling** : Go (service local), Docker multi-étapes, GitLab CI, Make, optimisation de builds.
- **Product & UX** : prototypage Figma, ateliers avec utilisateurs internes, plan de mesure produit.
- **Edge/IoT** : contraintes *on-edge*, anonymisation par événements, sensibilité au positionnement capteurs.

4.2 Méthodologiques et humaines

- Cadrage par problèmes, priorisation, **itération rapide** avec retours terrain.
- Communication avec intégrateurs et **équipe d'intégration** (vocabulaire commun, preuves par démos).
- Rigueur documentaire et **sens du détail** sur des tâches à fort impact opérationnel.

5 Conclusion

Ce stage m'a permis de **contribuer concrètement** à un levier critique du déploiement : l'annotation. La **refonte** livrée améliore l'expérience intégrateur (prévisualisation, édition complète, saisie assistée, précision) et prépare le terrain pour **mesurer objectivement** les gains de productivité. En parallèle, l'optimisation de la **chaîne CI/CD** et le renforcement de la **sécurité** de l'interface locale apportent des bénéfices transverses.